

Native Android XR Implementation Plan

End to end engineering blueprint for porting the existing Spectacles RelayView client to XREAL AURA while preserving the existing browser expert portal and Cloud Run relay.

Target

XREAL AURA / Android XR

Stack

Kotlin / Android Studio / Jetpack XR /
ARCore

Backend

Existing RelayView WebSocket relay

Prepared from the RelayView Technical Implementation Guide, current Android XR documentation, and the user's AURA Trusted Developer program communications. Private program details are summarized only where they affect implementation.

Version 1.0 | 1 October 2026

Implementation map

- 01 Architecture and non negotiables
- 02 Current AURA platform facts and program constraints
- 03 Repository and Android project structure
- 04 XR runtime, Full Space and capability gating
- 05 POV camera proof gate
- 06 Networking and existing RelayView protocol
- 07 Video capture, encoding and adaptive quality
- 08 Bidirectional audio pipeline
- 09 Remote click to world annotation
- 10 Frame pose synchronization for accurate annotations
- 11 Spatial UI and technician experience
- 12 Lifecycle, resilience and observability
- 13 Security, privacy and data handling
- 14 Testing matrix and acceptance criteria
- 15 Milestone execution plan
- 16 Gemini implementation contract
- 17 Risks, open questions and escalation checklist
- 18 Source register

01 Architecture and non negotiables

RelayView is already a decoupled remote assistance system: a wearable technician client streams POV video and two way audio through a stateless WebSocket relay, while a remote expert uses a standard browser to view the stream and place annotations. The Android XR port should replace only the technician runtime. The existing web portal and protocol remain the compatibility target.

Core rule: do not redesign the backend before achieving feature parity. First make AURA behave like another RelayView technician client.

Layer	Existing	AURA implementation
Technician runtime	Snap Spectacles / Lens Studio	Native Kotlin Android XR app
Transport	Secure WebSocket	Same secure WebSocket
Session pairing	Ephemeral 6 character code	Same handshake and code
Video	Base64 JPEG + frame terminator	Same V1 wire format
Voice	16 kHz PCM16 LE, binary tag 0x41	Same V1 wire format
Expert interaction	Normalized canvas x,y	Same message, later add optional frameId
World placement	InstantWorldHitTest	ARCore for Jetpack XR ray hit test
HUD	Lens Studio ScreenTransform	Jetpack XR spatial UI
Portal	Browser	Unchanged

Target topology

```
AURA technician
camera + microphone + head pose
↓
native Kotlin client
↓
wss://relayview.allthingskrazyy.com
↓
existing Node.js / Cloud Run relay
↓
existing browser expert portal
↑ annotation(x,y,label[,frameId]) + remote PCM audio
```

The original RelayView specification explicitly targets Spectacles, Quest and Android XR, and defines the web relay as pass through infrastructure with ephemeral session pairing, backpressure protection, binary audio relaying and diagnostic forwarding.

02 Current AURA platform facts and program constraints

Public Android XR documentation confirms XREAL Project Aura is treated as wired XR glasses and can be built with Android Studio and Jetpack XR. Current Jetpack XR setup guidance requires minSdk 24 and compileSdk 34 or higher. Google currently publishes device specific guidance and warns that XR capabilities can vary by device and mode.

Important correction to the earlier camera risk: current Android XR documentation now explicitly documents glasses hardware access through projected contexts for audio/display glasses, including camera and microphone permissions. AURA is a wired XR glasses target, so Gemini must still verify the exact AURA runtime path on hardware, but it should not assume camera access is impossible.

Program / platform finding	Engineering consequence
AURA Trusted Developer communications require current libraries/packages and minimal bugs	Pin versions deliberately, run release regression before every candidate
Android XR quality guidelines are the launch quality reference	Create a release checklist mapped to applicable XR quality items
XR immersive apps are generally expected to support both Galaxy XR and AURA unless hardware dependent	Avoid hard coding AURA assumptions in core modules
AURA EAP builds have had temporary maintenance / regressions	Log OS/build fingerprint in diagnostics and keep a known good build matrix
Spatial capability availability can change with runtime mode	Gate features dynamically rather than once at app startup
Full spatialization requires Full Space on wired XR glasses	RelayView should request Full Space for active assistance

Private program note

The user's Google AURA Trusted Developer emails indicate active program participation, recurring status reporting, launch quality expectations, and that at least one Gen 3 puck EAP build was temporarily withdrawn due to issues. Treat device build variability as a first class testing concern. Do not copy confidential program material into public repositories or app documentation.

03 Repository and Android project structure

Create a native Kotlin project with clear boundaries between transport, media, XR perception and UI. Gemini should implement interfaces first so hardware dependent code can be swapped without touching protocol logic.

```
app/src/main/java/com/allthingskrazyy/relayview/ MainActivity.kt RelayViewApplication.kt core/ AppConfig.kt
Result.kt xr/ XrSessionManager.kt XrCapabilityManager.kt FullSpaceController.kt HeadPoseProvider.kt perception/
PlaneTrackingManager.kt SpatialRaycastManager.kt AnchorManager.kt camera/ PovFrameSource.kt
AuraPovFrameSource.kt CameraMetadata.kt FrameEncoder.kt AdaptiveVideoController.kt audio/ MicrophoneCapture.kt
RemoteAudioPlayer.kt AudioJitterBuffer.kt AudioWatchdog.kt network/ RelayWebSocketClient.kt RelayProtocol.kt
SessionManager.kt ReconnectPolicy.kt annotation/ Annotation.kt AnnotationRenderer.kt FramePoseBuffer.kt
ViewportMapper.kt ui/ RelayViewApp.kt SessionPanel.kt DiagnosticsPanel.kt diagnostics/ Telemetry.kt
LatencyMonitor.kt BuildInfo.kt
```

- Use coroutines and Flow for state propagation.
- Keep MainActivity thin. It owns lifecycle and delegates.
- Use sealed state models for connection, call, camera and XR capability state.
- Do not let WebSocket callbacks directly mutate Compose UI.
- Keep protocol constants centralized and covered by unit tests.
- Create a hardware abstraction for POV capture before writing AURA specific code.

04 XR runtime, Full Space and capability gating

Create and retain the Jetpack XR Session per spatialized Activity. Current Android XR documentation warns that the Session is tied to Activity lifecycle and can become invalid when the Activity is recreated. Therefore session recreation must be a tested path, not an edge case.

Runtime sequence

Step	Action	Failure behavior
1	Launch 2D bootstrap UI	Never crash if XR is unavailable
2	Create XR Session off the critical UI path	Show initialization state
3	Request / enter Full Space for active assistance	Remain in safe panel mode if unavailable
4	Read current spatial capabilities	Enable only supported features
5	Configure ARCore perception and planes	Fallback annotations if perception unavailable
6	Start transport after permissions	Keep session code state independent from XR rendering

Capability gates

- Spatial UI: enable spatial technician panel only when supported.
- 3D content: required for world annotations. If absent, show annotation instructions in a 2D panel.
- Passthrough / perception: check at runtime.
- Plane tracking: use for accurate surface placement, otherwise forward projection fallback.
- Spatial audio: optional enhancement only, never a V1 dependency.

Do not cache capabilities forever. Re-evaluate when space mode or lifecycle state changes.

05 POV camera proof gate

This is the first hardware proof Gemini must complete on the actual AURA development kit. RelayView is useless as remote visual assistance if the application cannot obtain the technician's outward POV frame with sufficient fidelity and latency.

Proof gate A

Test	Pass condition
Permission	Camera permission can be requested and granted for the correct device context
Enumeration	App can identify the intended outward facing AURA camera
Capture	Continuous frames arrive without black / protected content
Metadata	Frame dimensions, timestamp, orientation and available intrinsics are recorded
Latency	Capture to encoded frame timing is measurable
Lifecycle	Disconnect, sleep, Activity recreation and resume recover cleanly

Implementation contract

```
interface PovFrameSource { suspend fun start(): Result<Unit> fun frames(): Flow<PovFrame> suspend fun stop() }
data class PovFrame( val id: Long, val timestampNanos: Long, val width: Int, val height: Int, val
rotationDegrees: Int, val bytesOrImage: Any, val cameraMetadata: CameraMetadata? )
```

For projected glasses hardware, current Android XR documentation describes using a projected context to initialize hardware services and requesting glasses specific CAMERA / RECORD_AUDIO permissions. On AURA, verify the actual wired XR API behavior and camera IDs on device. Never fabricate an undocumented API.

06 Networking and existing RelayView protocol

Use OkHttp WebSocket for V1 unless device testing exposes a concrete reason not to. Preserve the current relay contract byte for byte.

Message	Direction	Wire contract
Create session	AURA → relay	<code>{"action":"create-session"}</code>
Created	relay → AURA	<code>{"status":"created","sessionCode":"ABC-DEF"}</code>
Join session	web → relay	Existing portal behavior, unchanged
Annotation	web → AURA	<code>{"action":"annotate","data":{"x":0.48,"y":0.35,"label":"..."}}</code>
Audio	both directions	Binary packet, first byte 0x41, then PCM16 LE 16 kHz mono
Video	AURA → web	Base64 JPEG text terminated by <code> FRAME_END </code>
Mic status	AURA → web	Existing diagnostic JSON action

Connection policy

- Exponential reconnect with jitter, but preserve the current session only if the server contract supports reconnection semantics.
- Ping / RTT measurement independent from video traffic.
- Bound all outgoing queues. Video frames are disposable. Control messages are not.
- Never allow stale video frames to accumulate behind a slow socket.
- Parse text and binary frames on dedicated coroutine contexts.
- Surface server protocol errors in diagnostics with sanitized payload summaries.

Compatibility rule: the existing browser portal must work against AURA before any protocol extension is enabled.

07 Video capture, encoding and adaptive quality

V1 should deliberately retain RelayView's JPEG protocol. This gives the project a clean compatibility milestone. H.264 / WebRTC is a later optimization, not a prerequisite.

Profile	Scale	JPEG quality	Target FPS	Use
Crisp	0.85	72	20	Fine detail / good network
Balanced	0.65	60	15	Default
Fast	0.50	48	12	High RTT / weak link

Adaptive controller

RTT < 90 ms → CRISP 90–219 ms → BALANCED RTT ≥ 220 ms → FAST Additional downgrade signals: socket buffered bytes encoder time capture queue depth thermal state sustained frame drops Upgrade only after a stable hysteresis window.

- Use a single latest frame slot instead of an unbounded queue.
- Encode off the main thread.
- Record capture timestamp, encode duration, send timestamp and frame ID.
- Do not Base64 encode more than once.
- Validate portal rendering with rotation, aspect ratio and crop changes.
- Keep a future FrameEnvelope path ready, but do not break the current text frame terminator.

08 Bidirectional audio pipeline

RelayView's current audio contract is 16 kHz mono PCM16 little endian with a one byte 0x41 packet tag. Android maps cleanly to AudioRecord for capture and AudioTrack in streaming mode for playback.

Capture path

AURA microphone → AudioRecord → mono PCM → normalize / convert to signed 16 bit LE → prepend 0x41 → WebSocket binary

Playback path

WebSocket binary → verify packet tag 0x41 → PCM payload → bounded jitter buffer → AudioTrack MODE_STREAM → AURA output

Concern	Implementation
Audio source	Prefer VOICE_COMMUNICATION when supported and verify AEC behavior
Buffering	Start with ~40-100 ms jitter buffer and tune on hardware
Underrun	Track AudioTrack underruns and expose in diagnostics
Mic stall	Watch for prolonged zero capture while call is active
Call toggle	Prefer software transmit gating over repeated hardware churn
Focus / route	Handle Android audio focus and device route changes

Do not assume the exact Spectacles XR2 microphone lockup exists on AURA. Preserve the warm lifecycle pattern because it lowers restart latency, then validate its battery and thermal impact.

09 Remote click to world annotation

The expert clicks the video at normalized x,y coordinates. AURA must reconstruct the corresponding camera ray and hit test the tracked environment. The earlier conceptual `SpatialSession.hitTest(x,y)` shortcut should not be used unless the current SDK actually exposes such an API.

Required transform chain

```
portal click [0..1] → undo portal letterboxing / crop → image pixel coordinate → camera normalized coordinate →  
unproject using frame camera model / projection → transform ray by camera pose → ARCore for Jetpack XR hit test →  
select nearest valid tracked hit → create world locked annotation entity No valid hit: cameraPosition +  
cameraForward × 1.2 m
```

Selection rules

- Prefer physically plausible tracked surfaces in front of the technician.
- Reject hits behind the camera or beyond the configured assistance range.
- Use nearest valid hit unless product testing demonstrates a better heuristic.
- Preserve hit normal for marker orientation.
- Billboard only the readable label, not necessarily the physical pin geometry.
- Make annotation removal and clear all operations deterministic.

The fallback at roughly 1.2 m preserves the original RelayView behavior when world geometry is unavailable.

10 Frame pose synchronization for accurate annotations

This is the highest value architectural improvement over the current Spectacles protocol. A remote expert clicks an older video frame. If AURA raycasts from the technician's current head pose, the annotation can shift badly whenever the wearer moves between capture and click.

V2 compatible extension

```
Outgoing frame N: frameId = 18271 timestamp = ... cameraPose = ... projection / intrinsics = ... crop + rotation = ... Portal annotation: { "action": "annotate", "data": { "x": 0.42, "y": 0.31, "label": "Check connector", "frameId": 18271 } }
```

Maintain a rolling FramePoseBuffer keyed by frameId. When an annotation arrives, reconstruct the ray from the historical camera pose associated with the frame the expert saw. If frameId is absent, fall back to current pose so the old portal remains compatible.

State	Behavior
frameId found	Use historical pose and projection
frameId absent	Use current pose, V1 compatibility
frameId expired	Use current pose and log degraded placement
metadata unavailable	Fallback to calibrated display/camera mapping

This extension should be added only after basic AURA streaming and V1 annotations work.

11 Spatial UI and technician experience

RelayView is an industrial assistance tool, so the technician UI should be low clutter and glanceable. Preserve the existing Obsidian / Electric Lime / Slate / Sage design language.

State	Technician sees
Initializing	RelayView + device checks
Waiting for expert	Large session code, network state
Expert connected	Connection confirmation, call control
Call active	Small status HUD, RTT / quality only when useful
Annotation received	World pin + short label
Degraded mode	Clear camera / mic / perception status
Fatal error	Actionable recovery, never raw stack traces

UI state model

```
Initializing → PermissionRequired → Connecting → WaitForExpert(sessionCode) → Connected(sessionCode) → Calling(sessionCode, networkQuality) → Recovering(reason) → Error(recoverable, message)
```

- Keep central vision clear.
- Auto minimize passive status after inactivity.
- Use high contrast translucent surfaces where appropriate.
- Do not make diagnostics the default technician view.
- Keep the session code readable at a comfortable distance.
- Support controller / hand / system interaction without making a single input modality mandatory where avoidable.

12 Lifecycle, resilience and observability

AURA is an EAP / evolving device target. Robustness matters more than cleverness. Every subsystem needs explicit start, suspend, resume, reconnect and stop behavior.

Event	Expected behavior
Activity recreation	Recreate XR session and rebind render entities
Network loss	Stop sending stale media, reconnect with backoff
Expert disconnect	Return to waiting state without killing hardware unnecessarily
Camera failure	Keep audio/session alive, expose degraded state
Mic failure	Keep video/session alive, send mic-status diagnostic
AURA / puck disconnect	Release hardware resources and wait for reconnection
Thermal pressure	Reduce FPS / quality before failure
App background	Respect Android lifecycle and privacy indicators

Developer diagnostics panel

- OS build / device fingerprint and app version.
- XR runtime and library versions.
- Camera ID, resolution, FPS, rotation and capture latency.
- Encoder latency, JPEG quality, outgoing bitrate and dropped frames.
- WebSocket RTT, reconnect count and buffered bytes.
- Audio TX/RX rate, jitter depth, underruns and mic watchdog.
- Plane tracking state, hit test success rate and annotation count.
- Frame pose buffer age and historical pose hit rate.

Export diagnostics as a user initiated text report with no camera imagery or private session content.

13 Security, privacy and data handling

RelayView's architecture is intentionally ephemeral. Preserve that property. The relay should not become a media archive simply because the Android client is more capable.

- Use wss only. Fail closed on invalid TLS.
- Do not log raw audio, JPEG payloads, annotation labels or session codes in production telemetry.
- Keep session codes ephemeral and rate limit join attempts server side.
- Request camera and microphone permissions with clear purpose text.
- Display capture/call state clearly to the wearer.
- Do not persist camera frames to disk in production.
- Keep debug frame saving behind an explicit developer build flag.
- Sanitize server error strings before showing them in XR UI.
- Keep secrets and endpoints out of committed local config where credentials are involved.
- Review [Google Play Data Safety](#) declarations before release.

The existing design states zero payload storage and strict pass through relay behavior. Treat this as a product invariant unless the product explicitly changes.

14 Testing matrix and acceptance criteria

Do not call the port complete because it compiles. The acceptance suite must cover media, spatial accuracy, lifecycle and device build variation.

Area	Minimum acceptance
Session	Create code, portal joins, disconnect / reconnect recover
Video	Portal receives live AURA POV at stable ≥ 12 FPS in Fast profile
Audio TX	Expert hears intelligible technician audio without repeated stalls
Audio RX	Technician hears expert with bounded jitter and no runaway buffer
Annotation	Portal click produces visible world annotation
Placement	Known targets remain acceptably aligned during normal head movement
Backpressure	Network throttling increases drops, not multi second stale latency
Lifecycle	Resume and Activity recreation do not require app reinstall
Permissions	Denied camera / mic produces graceful degraded state
Device builds	Known good and current EAP build results recorded

Test scenarios

- Bright and dim environments.
- Close electronics bench and 2-5 m industrial target distances.
- Fast head turn immediately after frame capture.
- Wi Fi with injected latency and packet loss.
- Expert joins late, leaves, rejoins.
- Camera permission denied then granted.
- Microphone route change.
- Device / puck reconnect.
- Long 30 minute call for thermal and memory behavior.
- Portal on Chrome and Safari if both are current supported targets.

15 Milestone execution plan

Milestone	Deliverable	Exit gate
M0 Baseline	Project builds, dependency lock, diagnostics shell	APK runs on target
M1 XR	Session, Full Space, capabilities, basic panel	Stable XR lifecycle
M2 Camera proof	Real outward POV capture	Real frames + metadata
M3 Transport	Existing session handshake	Existing portal joins
M4 Video	JPEG stream to existing portal	Portal shows live AURA POV
M5 Audio	Two way PCM call	Stable bidirectional voice
M6 Perception	Planes + ray hit test	Local test ray hits
M7 Annotation	Remote click → world pin	Existing portal places pin
M8 Accuracy	FramePoseBuffer + optional frameld	Reduced motion error
M9 Resilience	Reconnect, lifecycle, thermal, diagnostics	30 min soak passes
M10 Release	Quality checklist, Play track build	Release candidate

Recommended implementation order

M2 is intentionally early. Camera proof is the critical dependency. Do not spend days polishing spatial UI before proving RelayView can access and stream the technician POV on the actual AURA hardware.

After V1 parity, evaluate MediaCodec H.264 and WebRTC only if measured JPEG bandwidth, CPU or latency justifies the migration. Keep WebSocket as signaling if WebRTC is introduced.

16 Gemini implementation contract

Paste this contract at the top of the Gemini coding task. It is designed to prevent hallucinated XR APIs and uncontrolled architecture changes.

You are implementing RelayView for XREAL AURA using native Kotlin, Android Studio, Jetpack XR and ARCore for Jetpack XR. NON NEGOTIABLES 1. Preserve compatibility with the existing RelayView web portal and `wss://relayview.allthingskrazzy.com`. 2. Do not invent Android XR, XREAL or camera APIs. Before using an XR API, verify it against the SDK version actually installed in this project. 3. Build incrementally. Each milestone must compile and run before starting the next. 4. First hardware gate: prove access to the actual outward AURA POV camera and capture real frames. 5. Keep camera, audio, network, perception, annotation and UI modules separate. 6. Existing V1 media protocol: video = Base64 JPEG + `|||FRAME_END|||` audio = binary 0x41 + PCM16 LE, mono, 16 kHz 7. Existing annotation protocol remains accepted: `annotate {x, y, label}` Add `frameId` only as an optional backwards compatible field. 8. Do not replace WebSocket/JPEG with WebRTC until V1 parity works. 9. Every hardware feature needs graceful failure and diagnostics. 10. Never silently substitute mock frames or simulated perception when running the hardware acceptance path. For every milestone: A. list exact dependencies and versions B. cite the SDK API used C. implement code D. compile E. report compile errors F. fix them G. provide hardware test steps H. do not proceed until the milestone passes.

17 Risks, open questions and escalation checklist

Risk / unknown	How to close it	Severity
Exact AURA outward camera API / IDs	Hardware proof on current build, inspect official samples	Critical
Camera frame ↔ XR pose calibration	Capture metadata + frame pose buffer + calibration tests	Critical
Raw camera intrinsics availability	Inspect camera characteristics / XR APIs; document fallback	High
EAP OS regressions	Record build fingerprint and known good matrix	High
Thermal cost of JPEG + audio + XR	30 min soak, adaptive quality	High
Activity / XR Session recreation	Automated lifecycle test path	High
Portal aspect ratio mismatch	Explicit crop/letterbox transform	High
Audio AEC / routing behavior	VOICE_COMMUNICATION tests on hardware	Medium
Cross device Galaxy XR compatibility	Avoid AURA only assumptions in core logic	Medium

Escalate to AURA / Android XR developer support with a minimal reproducible sample when any of these are blocked: outward camera cannot be opened with granted permission; camera frames do not expose enough geometry to map portal clicks; XR pose and camera timestamps cannot be correlated; or a reproducible EAP build regression breaks a previously working path.

When reporting an issue include device model, puck generation, OS build fingerprint, app commit, Jetpack XR versions, minimal logs, exact reproduction steps and whether the same APK worked on the previous build.

18 Source register

Project source

- RelayView Technical Implementation & Cross Platform Specification v1.0, October 2026. Used for system topology, protocol framing, adaptive JPEG profiles, audio contract, spatial annotation semantics, design language and scaling roadmap.

Current public Android XR references checked 1 October 2026

- Android Developers: Set up the Jetpack XR SDK.
<https://developer.android.com/develop/xr/jetpack-xr-sdk/set-up-sdk>
- Android Developers: Access a Session for spatialized UI and entities.
<https://developer.android.com/develop/xr/jetpack-xr-sdk/add-session>
- Android Developers: Check for spatial capabilities.
<https://developer.android.com/develop/xr/jetpack-xr-sdk/check-spatial-capabilities>
- Android Developers: Home Space to Full Space.
<https://developer.android.com/develop/xr/jetpack-xr-sdk/transition-home-space-to-full-space>
- Android Developers: Use a projected context to access glasses hardware.
<https://developer.android.com/develop/xr/jetpack-xr-sdk/access-hardware-projected-context>
- Android Developers: Request hardware permissions for audio/display glasses.
<https://developer.android.com/develop/xr/jetpack-xr-sdk/request-hardware-permissions>
- Android Developers: ARCore for Jetpack XR release notes.
<https://developer.android.com/jetpack/androidx/releases/xr-arcore>
- Android Developers: Android XR Catalyst, including XREAL Project Aura device class and supported toolchains. <https://developer.android.com/develop/xr/catalyst>

Private program sources reviewed

- Google Android XR / AURA Trusted Developer program communications, August–September 2026.
- September 16 launch submission reminder: current packages, XR quality guidance, cross device expectations and release track guidance.
- August 21 EAP notice: temporary withdrawal of an Aura Gen 3 puck build after issues were observed.
- Biweekly program communications: readiness tracking, technical support and active development status expectations.

Source handling note

Private program emails were used only to strengthen implementation and release planning. Confidential wording, links and distribution details are intentionally not reproduced in this document.

Implementation status legend

Label	Meaning
Confirmed	Supported by current project spec or public Android XR documentation
Hardware proof required	Must be validated on the actual AURA kit / current EAP build
V2	Useful enhancement after V1 protocol parity

END OF IMPLEMENTATION PLAN